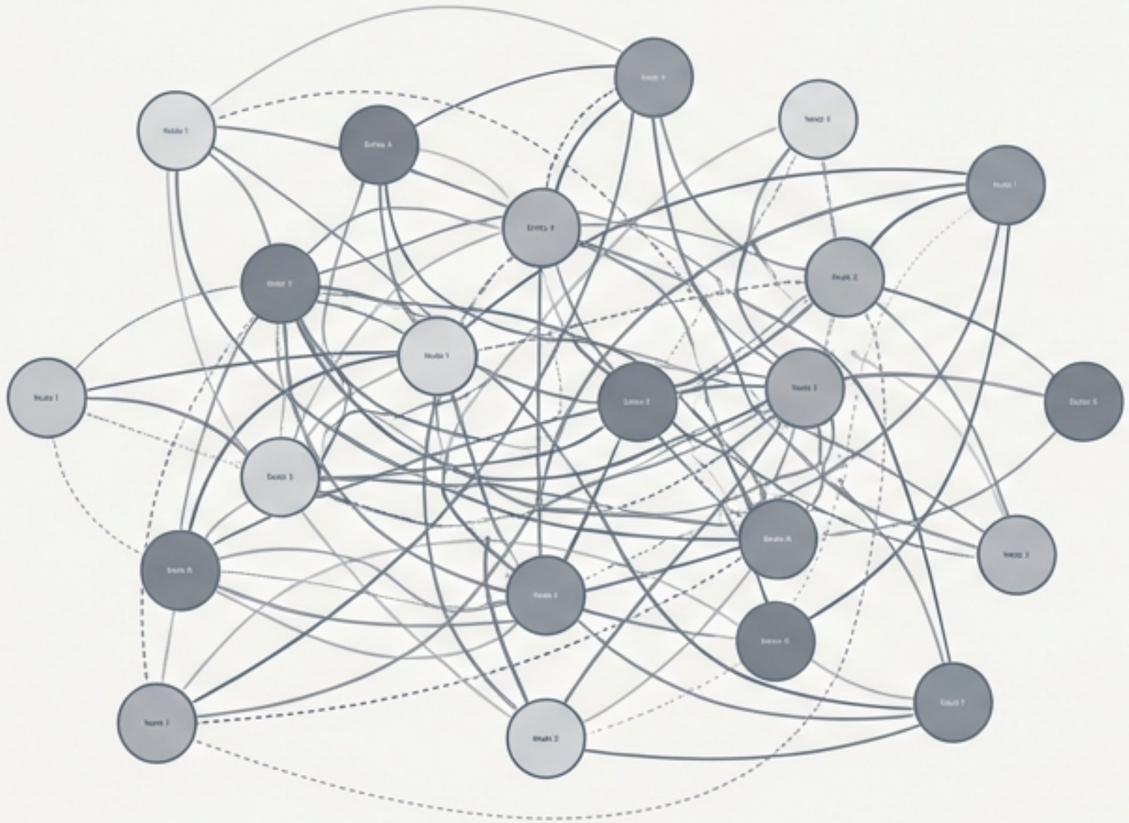
Code Representations for Semantic Graphs

Moving from Raw Data to Typed Architecture for Robust Testing

RAW DATA GRAPH



TYPED ARCHITECTURE

SemanticGraphNode

```
class SemanticGraphNode {
  id: string;
  type: NodeType;
  properties: Map<string, any>;

  // Edges are strongly typed
  outgoingEdges: List<Edge<this, any>>;
}
```

Edge<Source, Target>

```
class Edge<S extends SemanticGraphNode, T
extends SemanticGraphNode> {
  source: S;
  target: T;
  label: EdgeLabel;
  weight: double;
}
```

GraphProcessor

```
interface GraphProcessor {

  // Transforms raw graph into typed representation
  process(rawGraph: SeeDataGraph): TypedArchitecture;
  validate(typedGraph: TypedArchitecture);
}
```

TypedArchitecture

```
class TypedArchitecture {
  nodes: Map<String, SemanticGraphNode>;
  edges: List<Edge<any, any>>;
  scheme: GraphSchema;
}
```

Executive Summary: The Thesis

Tests should operate on stable code interfaces, not fragile text or raw graph structures.

Graph-to-Code Compilation

Transforming ontology nodes into executable classes to enforce structure.

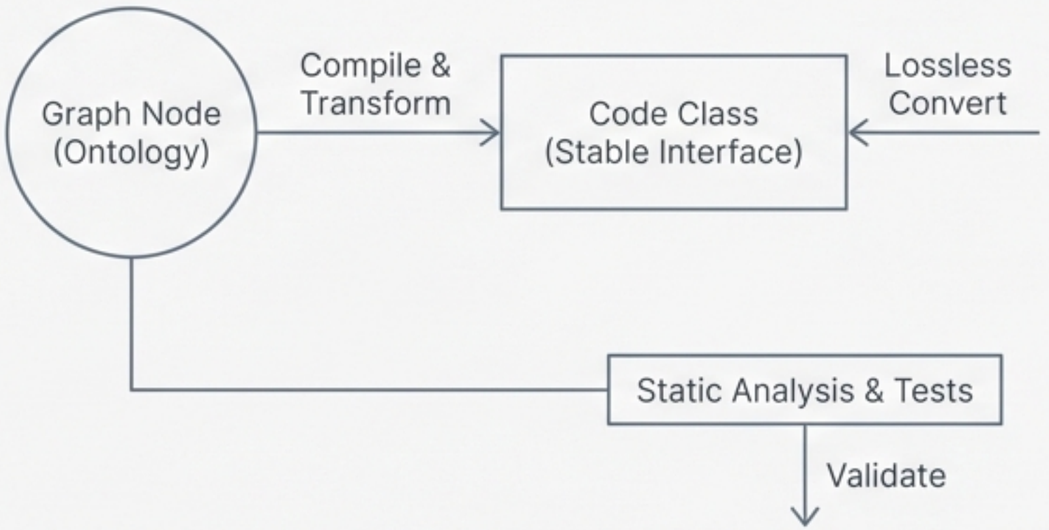
Type Safety & Analysis

Leveraging static analysis to catch errors before runtime.

Lossless Round-Trip

Seamless conversion between efficient graph storage and ergonomic code editing.

```
// Example: Ontology Node as a Class
@NodeEntity
public class Person implements GraphNode {
    @Property
    private String name;
    @Relationship(type = "WORKS_FOR", direction = Relationship.OUTGOING)
    private Company employer;

    // Constructor and methods for safe interaction
    public Person(String name) {
        this.name = name;
    }
    public void setEmployer(Company employer) {
        this.employer = employer;
    }
}
```



The Friction of Testing Semantic Data

The Text-Matching Trap

```
assert "remediation" in text
```

FRAGILE

- Fails on synonyms (e.g., ‘mitigation’)
- Fails on phrasing changes
- Tests grammar, not logic

The Graph Traversal Swamp

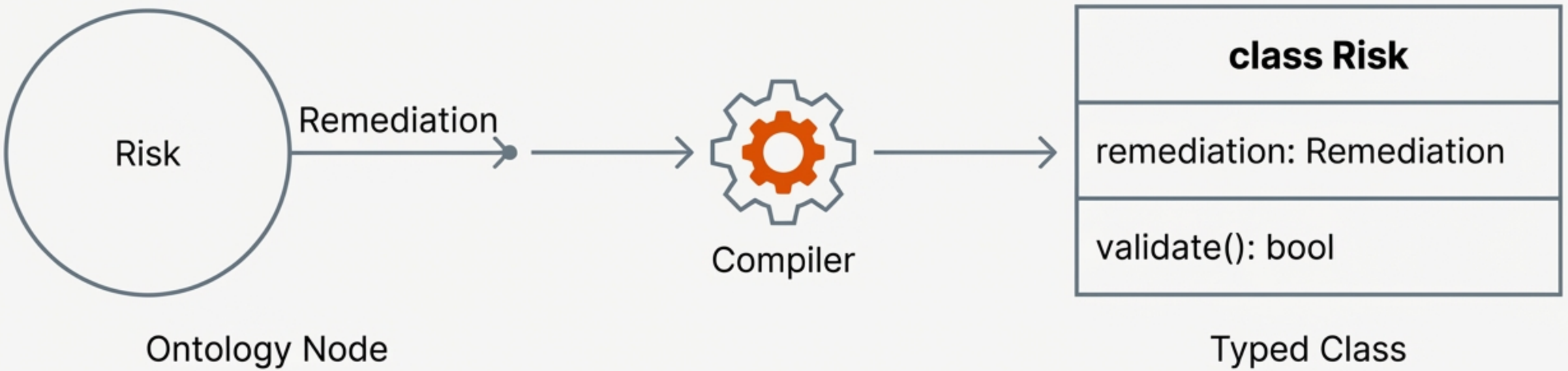
```
edges = [e for e in node.edges
   if e.type == "remediation"]
```

OBSCURE

- Verbose traversal logic
- Hides business intent
- Hard to refactor

The Solution: The ‘Code Representation’ Model

The graph *is* the code. Meaning becomes something the compiler understands.



- Nodes → Classes
- Edges → Typed Properties
- Constraints → Validation Methods

Mechanism: Nodes Become Classes

Every node type in the Ontology is compiled into a specific class definition. All generated classes inherit from a common base, ensuring consistent behaviour across the graph.

```python
class Risk(Semantic_Node):
    """
    A potential negative outcome.
    Derived from Ontology: v1.0
    """

    def __init__(self, ...):
        super().__init__(...)
        # Node initialization logic
```

Inherits core graph capabilities

Carries semantic metadata

Mechanism: Edges Become Typed Properties

Raw Graph Approach (High Cognitive Load)

```
# Finding a remediation for a risk
edges = graph.get_edges(source=risk_id, type="remediation")
if edges:
    remediation = graph.get_node(edges[0].target)
```

vs

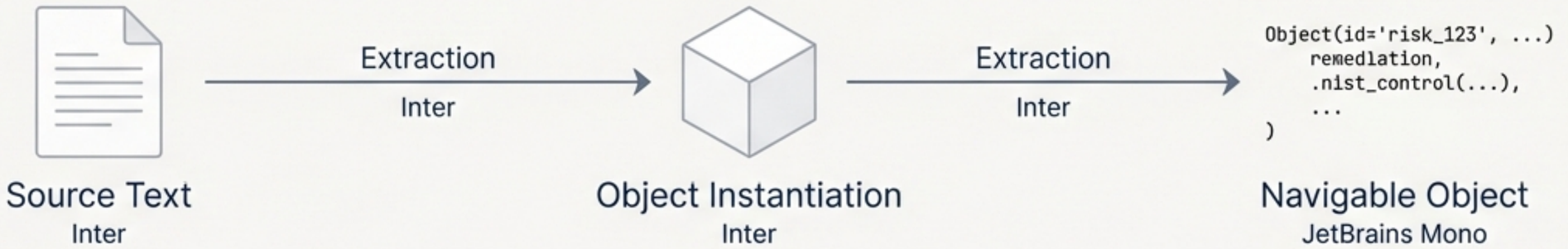
Typed Code Approach (Natural Language)

```
class Risk(Semantic_Node):
    remediation: Optional[Remediation]
    vulnerabilities: List[Vulnerability]

# Usage
my_remediation = risk.remediation
```

Explicit Schema Definition

Dot notation navigation

Runtime: Instantiation and Navigation

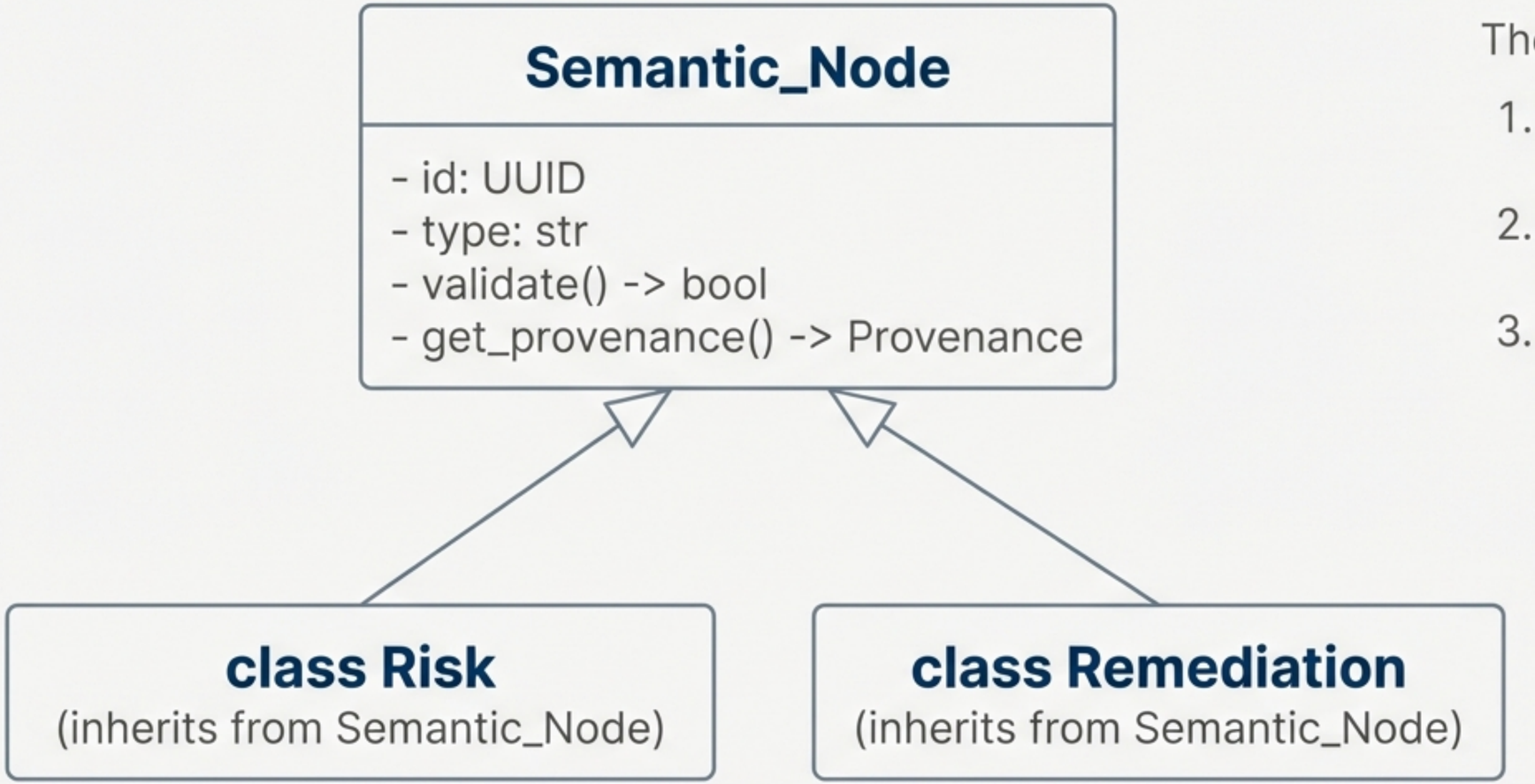



```python
# Traversing the graph through object properties

control_id = risk.remediation.nist_control.id

print(control_id)
# Output: "IA-2"
```

Complex graph queries become simple, readable object chains.

Inter Regular, Light Grey

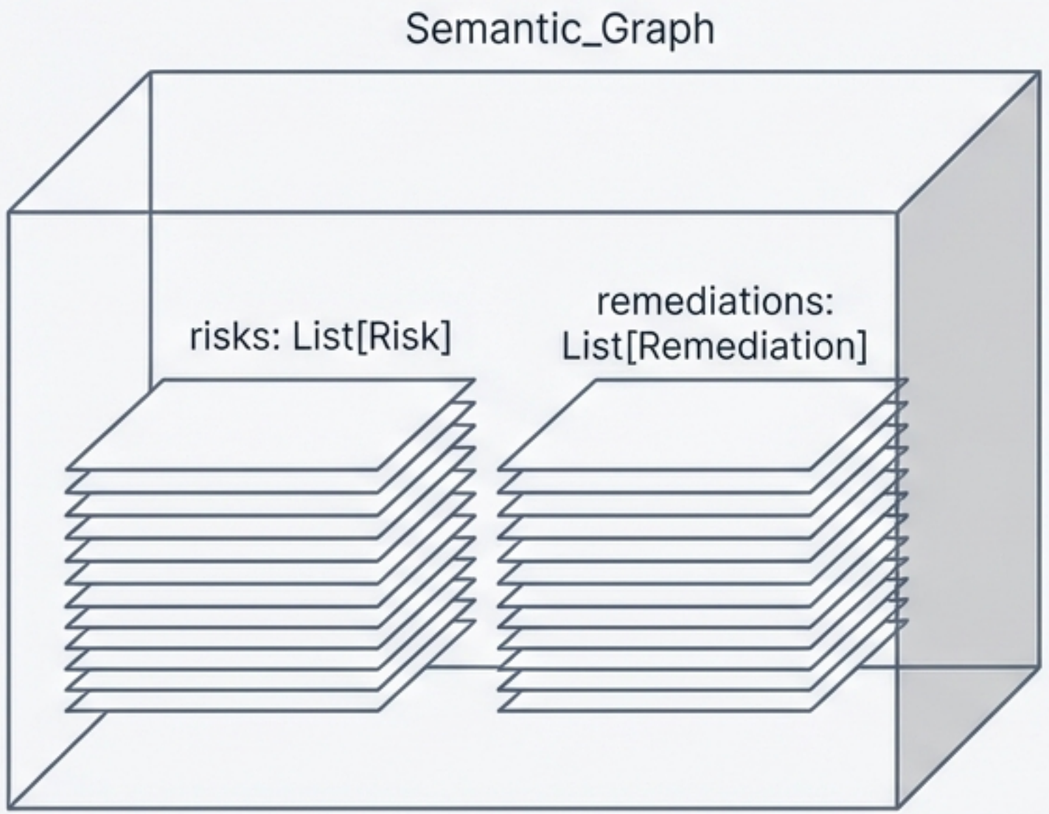
The Base Architecture: Semantic_Node



The Workhorse of the System:

1. Holds core identity and provenance.
2. Contains shared logic for all nodes.
3. Enforces baseline validation rules.

The Container: Semantic_Graph

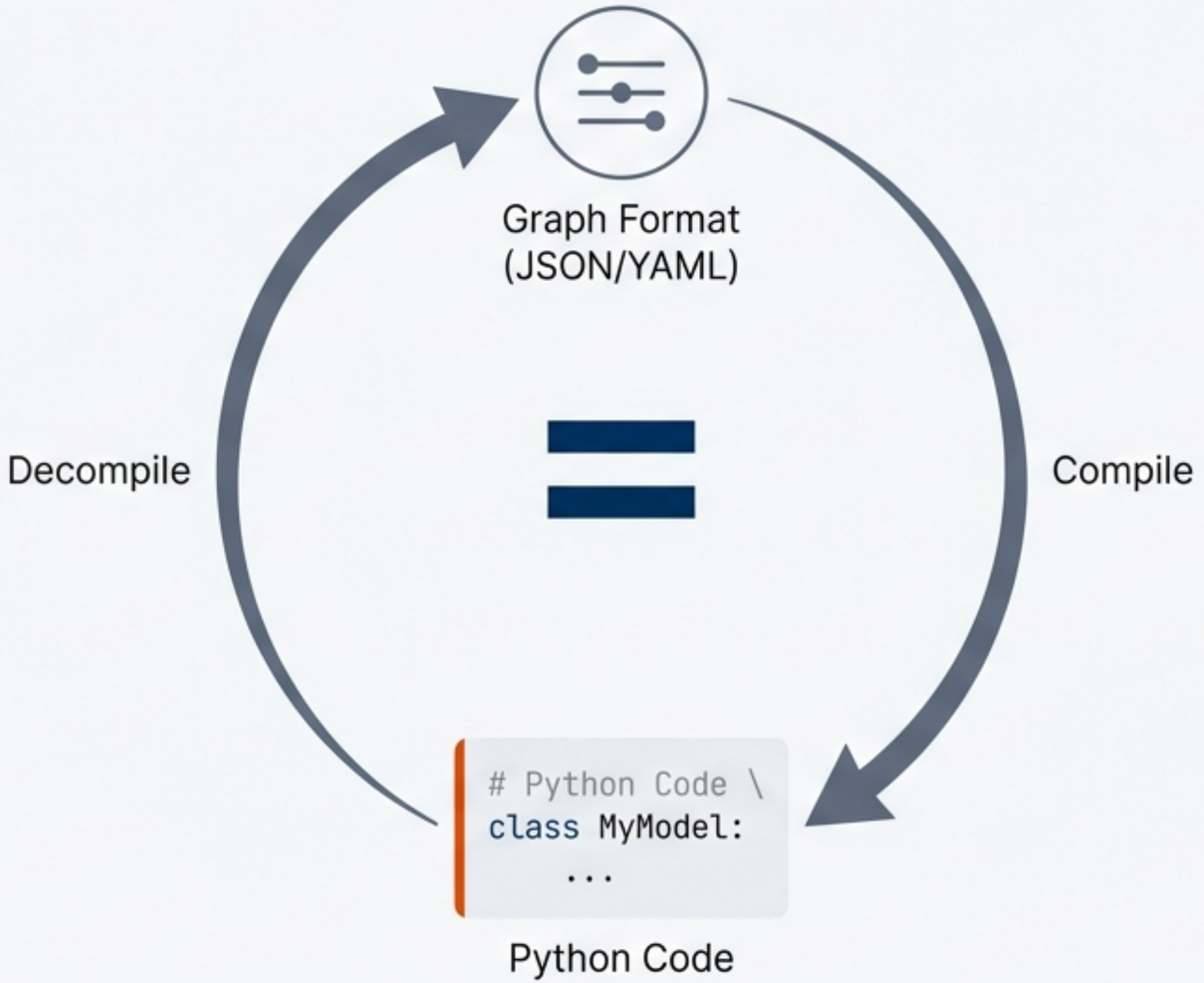



```python
# Initializing the wrapper
graph = Semantic_Graph(nodes)

# Typed entry point avoids generic searching
first_risk = graph.risks[0]
total_remediations = len(graph.remediations)
```

Acts as a typed wrapper, providing a structured entry point into the dataset.

The Round-Trip Guarantee

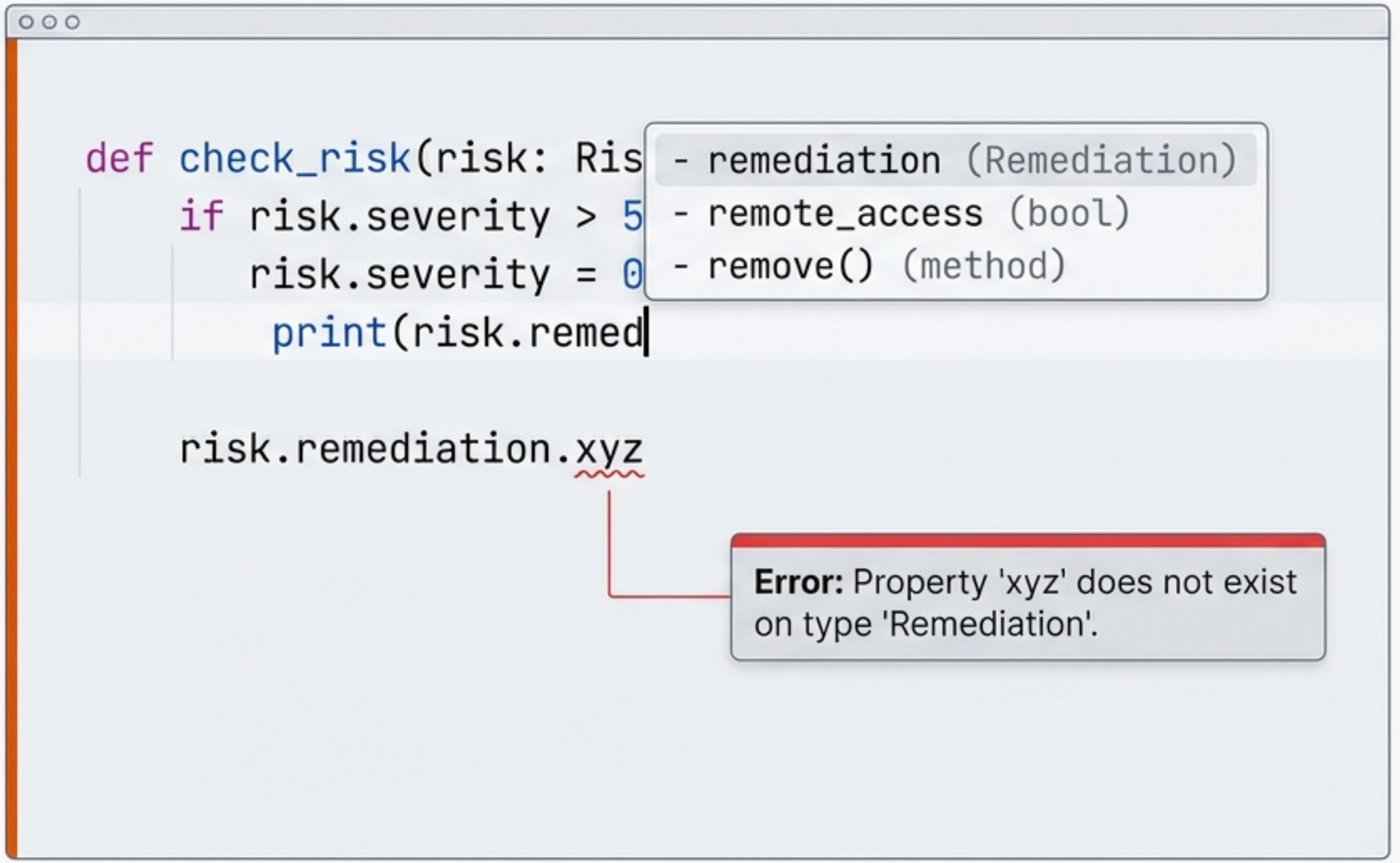
Lossless conversion enables flexible workflows.



Workflow Capabilities:

- Edit in Code
  (Leverage IDE power)
- Persist in Graph
  (Storage efficiency)
- Migrate seamlessy between representations

Developer Ergonomics & Static Analysis



1. Autocomplete discovery.
2. Compile-time error catching.
3. Instant refactoring.

Inter Regular, Light Grey

Advanced Patterns: Logic and Validation

Computed Properties

Calculating derived values on the fly.

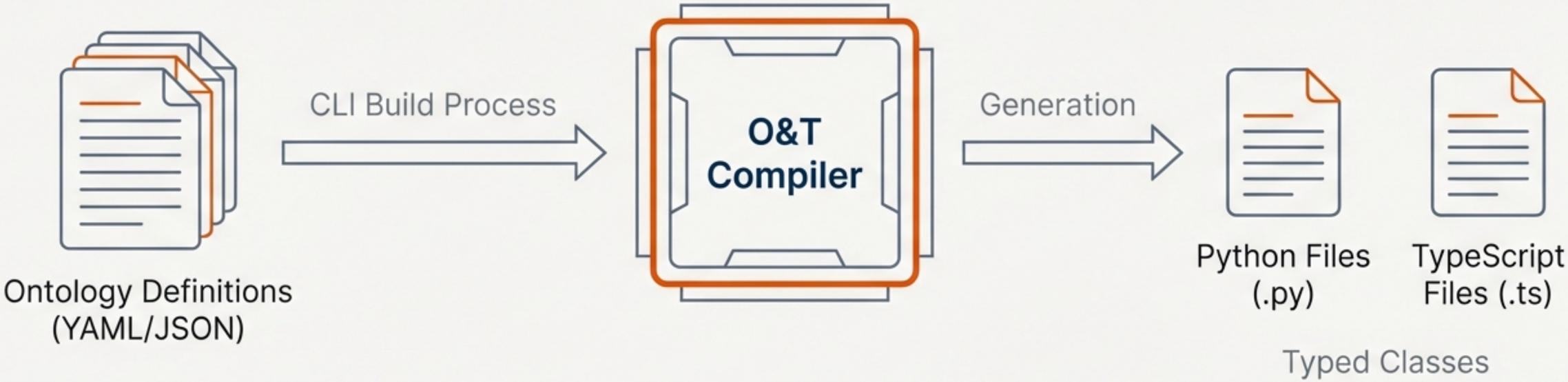
```python
@property
def severity(self) -> int:
    # Compute based on connected vulnerabilities
    return max([v.score for v in self.vulnerabilities])
```

Validation Methods

Enforcing Ontology constraints as executable code.

```python
def validate(self):
    if not self.remediation:
        raise ValidationError("Risk must have a Remediation")
```

The O&T Compiler Engine



Ensures code representation never drifts from the official Ontology definition.

The Revolution: Testing Meaning, Not Text

The Old Way (Fragile)

```
assert "remediation" in text_corpus
```

Breaks if word changes to “Mitigation”.

The New Way (Robust)

```
assert risk.remediation is not None
```

Passes regardless of wording, as long as structure exists.

Tests are now resilient to linguistic variation.

Architectural Decisions Log

ID	DECISION	RATIONALE
CR1	Nodes become classes	Enables type checking & IDE support.
CR2	Edges become properties	Natural dot-notation navigation.
CR3	Round-trip compilation	Lossless conversion for storage.
CR5	Validation from O&T	Constraints compile to runtime checks.
CR7	Tests target code	Stability across linguistic changes.

“Bridging the gap between unstructured human language and structured software engineering.”